

2026 年 7 月 9 日

已通读豆包 AI 翻译的文档，仍有一些不解之处，例如：

1. 第 6 部分的常驻空间上界 $N + (2r - 1) + 2(r - 1)B + B = N + O(rN^{1/r})$ 的这个系数 2 是什么
2. 下标访问原生 $O(r)$ 最坏时间是为什么，按我的理解顶多是二级指针， $O(1)$ 复杂度
3. 第 6.2 部分信用额度是什么鬼第一次听说，再看看
4. 第 8 部分看得有点云里雾里的，怀疑 AI 有删减

此外，还有一种局部看得懂，整合起来又有点晕晕的奇妙感觉

大致搞懂了它的数据结构构造（大成功？），复杂度分析也大致明白了

趣事：第 6.1 部分的推论 6.2 中我还在想 $r = \log N$ 时常驻空间为什么不是 $O(\log N \cdot N^{1/\log N})$ ，突然意识到 $N^{1/\log N}$ 是个常数

以下为 AI 翻译的文档：

最优可变长数组

Robert E. Tarjan[†] Uri Zwick[‡]

2026 年 7 月 9 日

摘要

可变长数组支持在数组尾部（或两端）增删元素，同时能够根据下标常数时间访问任意元素。由于数组元素数量会动态变化，要实现空间高效的可变长数组，必须配套动态内存管理机制。经典**倍增扩容**方案可实现规模为 N 的数组仅占用 $O(N)$ 空间，且单次操作均摊时间 $O(1)$ ，甚至最坏时间 $O(1)$ 。Sitarski、Brodnik 等人提出了更优方案：存储规模 N 的数组仅需 $N + O(\sqrt{N})$ 空间，单次操作仍保持 $O(1)$ 时间；Brodnik 等人还给出证明，该空间复杂度是理论下界。

本文区分两类空间开销：一是数组常态存储、元素访问所需的常驻空间；二是扩容/缩容操作过程中临时占用的额外空间。对任意整数 $r \geq 2$ ，我们证明：若扩容、缩容时可短暂使用 $N + O(N^{1-1/r})$ 临时空间，则常态存储规模 N 的数组仅需 $N + O(N^{1/r})$ 常驻空间；下标访问最坏时间 $O(1)$ ，增删操作均摊时间 $O(r)$ 。

我们通过一套抽象**扩容博弈**模型做精确分析，证明：若某类通用数据结构仅允许 $N + O(N^{1/r})$ 常驻存储空间，即便只支持尾部添加与下标查询，尾部新增操作的均摊代价下界也为 $\Omega(r)$ 。除非 $r = 2$ ，否则增删操作无法做到最坏时间 $O(1)$ 。

1 引言

数组与可变长数组是应用最广泛的数据结构。令人意外的是，本文将证明，关于它们仍存在大量未被充分挖掘的理论结论。我们提出一套简洁的全新可变长数组实现方案，多数场景下内存利用率优于此前所有设计。本文视角偏向理论，但给出的实现方案也具备实际工程价值。

规模为 N 的数组是存储序列 $a_0, a_1, \dots, a_{N-1}$ 的抽象数据结构：对任意 $0 \leq i < N$ ，下标 i 对应元素可在常数时间读取/修改。数组最基础的实现方式是连续内存块，每块内存存储一个元素或元素指针，依托机器随机访问能力实现常数时间读写；但原生数组的容量固定。

可变长数组（又称动态数组、动态表）允许数组容量动态增减，同时保证任意下标元素可常数时间读写。单纯缩小数组很简单：只需不再使用已分配内存的后半段，但这种方式内存利用率极低；扩容则更棘手——数组连续内存块末尾的内存大概率已被其他数据占用，因此可变长数组必须依赖动态内存管理。

为简化讨论，本文大部分内容仅讨论**单端可变长数组**（仅在数组尾部增删元素）：数组规模从 N 增至 $N+1$ 时，在末尾新增元素 a_N ；规模从 N 缩减至 $N-1$ 时，丢弃末尾元素 a_{N-1} 。这类可变长数组是栈的泛化结构。本文所有结论均可拓展至两端均可增删的双端可变长数组（即双端队列 deque）：只需用两个单端可变长数组拼接实现即可。

教材中经典的可变长数组实现是**倍增扩缩容**（见 Cormen 等人《算法导论》17.4 节）：初始分配一块定长内存；内存填满时，分配一块两倍大小的新内存，将全部元素拷贝过去并释放旧内

存；当内存使用率低于 $1/4$ 时，分配一半大小的新内存拷贝元素、释放旧内存。均摊分析可证明，单次扩容/缩容操作均摊代价 $O(1)$ ，元素读写最坏时间 $O(1)$ ；借助后台渐进重构技术，甚至能将扩缩容优化为最坏时间 $O(1)$ 。

维基百科记载，Java `ArrayList`、Python `PyListObject`、C++ `STL vector` 等主流容器均采用几何扩缩容策略，扩容系数取 1.25×2 。

倍增方案仅保证 $O(N)$ 空间复杂度，虽能满足多数场景，但纯扩容场景下最多会浪费 50% 已分配内存。更一般地，若扩容系数为 $1 + \alpha$ ，内存闲置比例可达 $\frac{1}{1+\alpha}$ ；若缩小 α 降低闲置占比，单次扩容均摊代价会上升至 $O(\frac{1+\alpha}{\alpha})$ 。

一个核心问题：能否构造仅占用 $N + o(N)$ 空间、且所有操作 $O(1)$ 时间的可变长数组？Sitariski 与 Brodnik 等人给出肯定答案：设计实现仅需 $N + O(\sqrt{N})$ 空间，我们将在第 3 节详细介绍。Brodnik 等人同时证明该方案存在下界——任意可变长数组实现，在某些时刻必然占用 $N + \Omega(\sqrt{N})$ 空间，第 4 节会复述其简洁证明。

本文核心创新：区分**常驻存储空间**（常态存放数组、支持下标读写所需空间）与**扩缩容临时空间**（扩容/缩容过程中短暂占用的额外内存）。很多场景下，我们愿意牺牲少量临时内存，换取常态存储更高的紧凑度；例如程序中存在大量可变长数组，但同一时刻仅一个数组执行扩缩容。

我们首个突破 Brodnik 等人下界的实现方案：常态存储仅 $N + O(N^{1/3})$ 空间，仅在扩缩容时临时占用 $N + O(N^{2/3})$ 空间；元素读写最坏时间 $O(1)$ ，增删均摊 $O(1)$ 。同时我们证明该方案渐近最优：任何仅用 $N + O(N^{1/3})$ 常驻空间的实现，扩缩容时必然需要 $N + \Omega(N^{2/3})$ 临时空间。

更一般地，对任意 $r \geq 2$ ：

1. 常态存储开销： $N + O(N^{1/r})$ ；
2. 扩缩容临时开销： $N + O(N^{1-1/r})$ ；
3. 下标访问最坏时间 $O(1)$ ；
4. 尾部增删操作均摊时间 $O(r)$ 。

由于无法使用后台重构技术，除非 $r = 2$ ，否则 $O(r)$ 的均摊代价无法优化为最坏时间 $O(1)$ 。 $r = 2$ 对应 Sitariski、Brodnik 的经典方案。

1.1 论文章节结构

1. 第 2 节：精确形式化问题定义与计算模型；
2. 第 3 节：梳理前人相关工作，重点介绍 Sitariski 的 HAT 结构、Brodnik 等人的方案；
3. 第 4 节：复述 Brodnik 的下界证明，并给出我们的拓展下界；
4. 第 5 节：给出 $r = 3$ 的完整构造（常驻 $O(N^{1/3})$ 、临时 $O(N^{2/3})$ ）；
5. 第 6 节：对任意 $r \geq 2$ 给出通用无递归实现，证明增删均摊 $O(r)$ ；
6. 第 7 节：提出一套结构变换技巧，将常驻空间从 $O(rN^{1/r})$ 优化至 $O(N^{1/r})$ ，同时不破坏常数访问、 $O(r)$ 均摊增删；

7. 第 8 节：抽象出**扩容博弈**模型并完整求解，用于证明下界；
8. 第 9 节：依托扩容博弈，证明仅使用 $N + O(N^{1/r})$ 常驻空间时，尾部新增操作均摊代价下界 $\Omega(r)$ ；
9. 第 10 节：总结与展望。

2 可变长数组形式化定义

可变长数组作为抽象数据类型，支持以下操作：

1. `Array()`：创建空数组并返回；
2. `A.Length()`：返回数组当前元素个数；
3. `A.Get(i)`：返回下标 i 对应的元素 a_i ，要求 $0 \leq i < A.Length()$ ；
4. `A.Set(i, a)`：将下标 i 元素修改为 a ，要求 $0 \leq i < A.Length()$ ；
5. `A.Grow(a)`：数组长度 $+1$ ，末尾新增元素 a ；
6. `A.Shrink()`：数组长度 -1 ，丢弃末尾元素。

本文仅允许在数组尾部增删元素。若支持任意位置插入删除（并同步平移后续元素下标），问题难度会大幅提升；此时所有操作最优时间界为 $O(\frac{\log N}{\log \log N})$ ，且该界紧（Dietz、Fredman&Saks）。另有一类方案：下标访问最坏 $O(r)$ ，插入删除均摊 $O(N^{1/r})$ （Goodrich、Joannou 等人）。再次强调：本文仅讨论尾部增删场景。

我们假设内存管理器支持分配、释放任意长度的定长数组（类比 C/C++ 的 `malloc/free`），简化起见，单次分配/释放操作代价为常数；即便分配长度 N 的内存块代价为 $O(N)$ ，本文所有均摊复杂度结论依然成立。本文不讨论内存管理器底层实现。

可变长数组实现由若干动态分配的定长内存块（简称块）组成，分为三类：

1. 数据块：存储数组元素，每个元素占一个机器字；
2. 索引块：存储指向其他块的指针，每个指针占一个机器字；
3. 辅助块：存放数据结构元信息，如各块长度。

本文所有实现均遵循该框架。若数组当前规模为 N ，则全部数据块总容量至少为 N （第 4 节定理 4.1 证明后补充讨论）。除主索引块外，所有块必须被某索引块内的指针引用，否则无法访问。数据结构总空间为所有已分配块的长度之和。

$[(s(N), t(N)) \text{ 实现}]$ 设 $s(N), t(N)$ 为单调不减函数。若某可变长数组满足：

1. 存储规模 N 的数组时，常驻总空间不超过 $N + s(N)$ ；
2. 执行扩容/缩容操作过程中，瞬时总空间不超过 $N + t(N)$ ；

则称其为 $(s(N), t(N))$ 实现。

3 前人相关工作

设计空间高效可变长数组属于紧凑数据结构领域，Raman、Munro 等人已有大量综述，但仅有少数文献专门研究本文定义的单端可变长数组。

3.1 基础单块实现

基础实现仅用一块连续定长内存承载数组：内存填满时，分配更大内存、拷贝全部元素、释放旧内存；内存空置过多时，分配更小内存拷贝元素。实现可自由选择新内存块的尺寸。

1. 极致省常驻空间方案：每次增删都重新分配恰好等于元素数量的内存。常驻空间 $N + O(1)$ （仅存长度与内存指针），但单次增删均摊代价 $\Omega(N)$ ；扩缩容时需同时保存新旧两块内存，临时空间 $2N + O(1)$ ，仅适合极少扩缩容的场景。
2. 几何扩缩容（工程主流）：固定参数 $\alpha > 0$ ，内存填满时分配 $(1 + \alpha)N$ 大小新块；内存元素仅占当前块容量 $\frac{1}{(1+\alpha)^2}$ 时，分配 $\frac{N}{1+\alpha}$ 大小新块拷贝元素。简单计算可得：扩容均摊代价 $\frac{1+\alpha}{\alpha}$ ，缩容均摊代价 $\frac{1}{\alpha}$ 。合理选取 α 可平衡空间与时间，适配绝大多数工程场景。

3.2 Sitarski 的哈希数组（HAT）

Sitarski 提出名为 HAT（哈希数组树）的结构，但命名存在误导——不含哈希，本质分层链表。

HAT 维护长度为 B 的索引块，满足 $\sqrt{N} \leq B < 4\sqrt{N}$ ；配套 $\lceil N/B \rceil$ 或 $\lceil N/B \rceil + 1$ 个尺寸 B 的数据块。 N 个元素顺序存入前 $\lceil N/B \rceil$ 块； N 无法整除 B 时最后一块半满；存在 $\lceil N/B \rceil + 1$ 块则为空。索引块第 i 项存储第 i 个数据块指针。

总常驻空间： $N + 3B + 2 = N + O(\sqrt{N})$ 。 $3B$ 为索引块 + 两块闲置数据块开销。下标 i 元素位于第 $\lfloor i/B \rfloor$ 块、偏移 $i \bmod B$ ，读写最坏 $O(1)$ ； B 取 2 的幂时可通过位运算快速计算下标。

扩容逻辑： $\sqrt{N} < B$ 即 $N < B^2$ 时，末尾块未满足直接写入；已满但存在空块则写入空块；无空块则分配新 B 块。分配代价常数则扩容最坏 $O(1)$ ；分配代价 $O(B)$ 时扩容均摊 $O(1)$ 。

满载 $N = B^2$ 时将 B 翻倍重构；初始 B 为 2 的幂则重构后不变。重构 $O(N)$ 拷贝代价分摊至 $\Omega(N)$ 次扩容，均摊 $O(1)$ 。

缩容对称：仅末尾两块全空才释放尾部块； $B = 4\sqrt{N}$ 即 $N = B^2/16$ 时折半重构。两次重构间至少 $\Omega(N)$ 次操作，缩容均摊 $O(1)$ 。

重构仅需 $O(\sqrt{N})$ 临时空间，新旧块逐块分配释放，无需完整保存两份数组。

可通过双尺寸块 + 后台重构转为最坏时间版本，但复杂度大幅提升；下一节无需重构、元素不移动。

3.3 Brodnik 等人的分层块结构

Brodnik 独立提出分层方案，常驻 $N + O(\sqrt{N})$ ，扩缩容最坏 $O(1)$ （分配常数时间）；优势：无全局重构、元素不移动。两种变体：

1. 基础变体：数据块尺寸 $1, 2, 3, \dots$ ；前 k 块总容量 $\approx k^2/2$ ， $k = \lceil \frac{\sqrt{8N+1}-1}{2} \rceil$ ；末块半满，预留一块空块。索引块尺寸 $\Theta(\sqrt{N})$ ，自身朴素动态数组实现。有空位直接写入；无空位分配新块，

索引块满则扩容拷贝指针，后台拷贝实现最坏 $O(1)$ 。下标块编号 $\ell = \lceil \frac{\sqrt{8i+1}-1}{2} \rceil$ ，块内偏移 $i - \frac{\ell(\ell-1)}{2}$ ，读写 $O(1)$ 但需要开平方。

2. 无平方根变体：所有块尺寸为 2 的幂，引入超块，移位定位规避平方根运算。

4 Brodник 下界定理及其拓展

本节复述原始下界与分离空间拓展下界。任意仅支持扩容、下标访问的可变长数组，在某些时刻必然占用 $N + \Omega(\sqrt{N})$ 空间， N 为当前元素总数。

证明. 完成 N 次扩容后设使用 k 块连续内存；全部数据块容量至少 N ，每块需一条指针，指针总开销 k 。

- 若 $k \geq \sqrt{N}$ ：总空间 $\geq N + \sqrt{N}$ ，下界成立；
- 若 $k < \sqrt{N}$ ：必存在尺寸 $\ell > \sqrt{N}$ 的块。该块在数组规模 $N' \leq N$ 时分配，分配瞬间总空间 $N' + \ell > N' + \sqrt{N'}$ ，下界成立。

证毕。 □

下界两条基础假设：

1. N 个独立 w 比特元素至少占用 N 机器字；
2. k 条指针占用 k 机器字（地址空间 2^w ）。

地址空间 2^{cw} ($0 < c < 1$) 时指针开销为 ck ，仅常数变化，不改变阶。

$[(s(N), t(N))$ 分离下界] 任意仅支持扩容、访问的 $(s(N), t(N))$ 实现，满足 $s(N) \cdot t(N) \geq N$ 。

证明. N 个元素分散至多 $s(N)$ 块，鸽巢原理存在块尺寸 $\geq N/s(N)$ 。该块分配时刻瞬时额外空间 $\geq N/s(N)$ ； $t(N)$ 单调， $t(N) \geq N/s(N)$ ，变形得 $s(N)t(N) \geq N$ 。证毕。 □

对任意整数 $r \geq 1$ ，常驻仅 $N + O(N^{1/r})$ 的实现，扩容瞬时必有 $N + \Omega(N^{1-1/r})$ 额外空间。后文证明该下界紧。

5 $r = 3$ 的极简实现：($O(N^{1/3}), O(N^{2/3})$) 方案

构造常驻 $O(N^{1/3})$ 、临时 $O(N^{2/3})$ ，访问最坏 $O(1)$ ，增删均摊 $O(1)$ 。

核心改造 HAT：外层块 $N^{2/3}$ 、索引 $N^{1/3}$ ，半满块嵌套一层 HAT，嵌套额外 $O((N^{2/3})^{1/2}) = O(N^{1/3})$ 。合并拷贝瞬时 $\Omega(N^{2/3})$ 临时空间，匹配下界。也可基于 Brodник 分层构造。

无递归描述：参数 $N^{1/3} \leq B \leq 4N^{1/3}$ ，两类块：

1. 大块：尺寸 B^2 ，总数约 N/B^2 ，永久填满；
2. 小块：尺寸 B ，至多 $2B$ 个，至多一块半满、至多一块空。

两级索引分别管理大小块； $2B$ 小块满则合并为一块大块；无小块则拆分大块。所有访问最坏 $O(1)$ ，增删均摊 $O(1)$ 。

6 对任意 $r \geq 2$ 的通用 $(O(rN^{1/r}), O(N^{1-1/r}))$ 实现

泛化 $r = 3$ 构造，无递归，常驻 $O(rN^{1/r})$ ，临时 $O(N^{1-1/r})$ 。

参数 $N^{1/r} \leq B < 4N^{1/r}$ ，块尺寸 $B, B^2, \dots, B^{r-1}$ 。仅扩容每层最多 B 块；支持扩缩容放宽至每层 $2B$ 块（冗余基数 B 计数器）。

- $B^2, \dots, B^{r-1}$ 块永久填满；
- 仅末尾 B 块可半满。

扩容逻辑：末尾 B 块有空直接写入；满且不足 $2B$ 分配新 B 块；满 $2B$ 则 B 个同尺寸小块合并高一阶大块，等价基数 B 进位计数。

$N = B^r$ 时 B 翻倍全局重构；配套 $r-1$ 层索引。常驻上界 $N + (2r-1) + 2(r-1)B + B = N + O(rN^{1/r})$ ；合并 B^{r-1} 块临时峰值 $O(N^{1-1/r})$ 。

原生下标访问最坏 $O(r)$ ；6.3 节优化至 $O(1)$ 。增删均摊 $O(r)$ ，元素逐层向上合并，重构分摊常数开销。

缩容：每层上限 $2B$ 块；满 $2B$ 合并；无小块时拆分最小大块，均摊仍 $O(r)$ 。

6.1 6.1 实现伪代码说明

n_i ：尺寸 B^i 块数量； $A[i]$ 对应索引块； n_0 末尾 B 块已存元素。

6.1.1 Grow(a) 扩容流程

1. 若 $N = B^r$ ，执行 $\text{Rebuild}(2B)$ ， B 翻倍；
2. 若 $n_1 = 2B, n_0 = B$ （ B 块全满），调用 Combine-Blocks 合并；
3. 无空位则分配新 B 块， n_0 置 0；
4. 写入新元素， n_0, N 自增 1。

Combine-Blocks ：最小未满足层 k ，自底向上每层 B 小块合并一块大块，调整指针释放旧块。

6.1.2 Shrink() 缩容流程

1. 若 $N = (B/4)^r$ ，执行 $\text{Rebuild}(B/2)$ ， B 折半；
2. 无 B 块调用 Split-Blocks 拆分大块；
3. 删除末尾元素， n_0, N 自减 1； $n_0 = 0$ 则释放空 B 块。

Split-Blocks ：最小非空大块拆多层小块，分配拷贝释放原块。

辅助函数：

- $\text{Allocate}(\ell)$ ：分配长度 ℓ 块，返回指针；
- $\text{Deallocate}(X)$ ：释放块 X ；

- $\text{Copy}(X, x, Y, y, \ell)$: X 偏移 x 起 ℓ 元素拷贝至 Y 偏移 y ;
- $\text{Rebuild}(B')$: 以 B' 全局重构整套分层结构。

该实现为 $(O(rN^{1/r}), O(N^{1-1/r}))$ 可变长数组; 扩容缩容均摊 $O(r)$, 下标读写最坏 $O(1)$ 。
取 $r = \log N$, 存在实现常驻 $N + O(\log N)$, 增删均摊 $O(\log N)$, 访问最坏 $O(1)$ 。

6.2 6.2 增删均摊代价证明 (引理 6.3)

操作代价定义为元素拷贝赋值次数。

1. 扩容: 基础写入代价 1; 每个底层元素分配 $r - 1$ 信用, 每合并一层消耗 1 信用, 合并均摊 0, 单次扩容均摊 $O(r)$;
2. 缩容: 每次缩容为每层增加固定信用; 拆分拷贝由累积信用覆盖, 拆分均摊 0, 单次缩摊 $O(r)$;
3. 全局重构: 两次重构间至少 $\Omega(N)$ 次操作, 总拷贝 N 分摊, 不增加渐近阶。

综上扩缩容均摊 $O(r)$ 。

6.3 6.3 $O(1)$ 下标访问优化 (引理 6.4)

假设 CPU 内置常数 msb (最高置位查询), 改造结构:

1. $B = 2^b$ (2 的幂);
2. 总规模冗余 B 进制 $N = \sum_{j=0}^{r-1} n_j B^j$, $0 \leq n_j \leq 2B$;
3. 块计数压缩存入两机器字 N^0, N^1 , 移位按位提取;
4. 预计算前缀容量 N_k , 表示尺寸 $\leq B^{k-1}$ 总元素。

访问下标 i 步骤:

1. 末尾偏移 $x = N - 1 - i$;
2. msb 定位进制最高位, 确定块阶 k ;
3. 移位按位运算计算块内偏移, 全程常数操作。

无原生 msb 时复杂度 $O(\log \log w)$, 不改变渐近; 工程 r 极小, 遍历 r 层也实用。

7 数据结构变换优化常驻空间

$r(N)$ 随 N 缓慢增长时, 将常驻 $O(rN^{1/r})$ 降至 $O(N^{1/r})$, 维持 $O(r)$ 均摊、 $O(1)$ 访问。[标准 $(s(N), t(N))$ 实现] 满足两条:

1. 分配新块时顺序拷贝旧块, 拷贝完成释放旧块;
2. 拷贝结束常驻回到 $N + s(N)$ 。